



Proyecto 1

Creación de un Chat de Texto TCP

Sánchez Girón Yael Adrian¹

¹Facultad de Ciencias, UNAM

Resumen

Este proyecto presenta el desarrollo de un sistema de chat basado en la arquitectura Cliente-Servidor, diseñado para operar mediante conexiones y manejo de red. Ante la restricción estricta de no utilizar el lenguaje Java, se adoptaron tecnologías alternativas de alto rendimiento tras una ardua investigación. El servidor fue implementado en Rust, aprovechando su gestión de memoria sin recolector de basura (*Rust Ownership*) y garantizando tolerancia a fallos mediante la ejecución asíncrona de peticiones. Por su parte, el cliente se desarrolló utilizando el *framework* Flutter (Dart) bajo el patrón de diseño Modelo-Vista-Controlador (MVC), proporcionando una interfaz gráfica nativa para entornos de escritorio (Linux, macOS). El resultado es un sistema funcional, escalable y estructurado que procesa y traduce interacciones en lenguaje natural a un protocolo estandarizado basado en JSON.

1. Introducción

La materia de modelado y programación, además de ser una materia formativa, se considera como una materia la cual tiene como objetivo obtener la experiencia para integrar las habilidades de programación y modelado aplicando los conocimientos adquiridos en los primeros semestres de la licenciatura de Ciencias de la Computación. Bajo este contexto, es que se presenta este primer proyecto de Modelado y Programación, el objetivo de este proyecto es el poder desarrollar desde 0 un sistema de chat funcional donde múltiples clientes solicitan servicios desde un hilo de ejecución, empleando programación asíncrona. Para la construcción del servidor se optó por utilizar el lenguaje de programación Rust, esto debido a como es que este lenguaje maneja la memoria, además de que en el contexto actual se está iniciando a re-escribir código muy importante en este lenguaje de programación, como bien podría ser todo el OS de Windows, o las pruebas que se están realizando en el kernel de Linux. En cuanto al cliente, se optó por utilizar el framework de Flutter, el cual es un proyecto Open-Source realizado por Google, siendo OOP, ideal para construir interfaces gráficas. El alcance del proyecto contempla el diseño estructural de tanto el Server, como del Cliente. El server

asume el rol de establecer conexiones duraderas, almacenar el estado de los usuarios y transmitir mensajes de forma concurrente y asíncrona. En contraste, el cliente asume la responsabilidad de traducir los eventos físicos del usuario en comandos empaquetados, aislándolo completamente de la complejidad técnica de la red. Ambos extremos se comunican mediante un protocolo estructurado que garantiza que cualquier cliente de la clase pueda conectarse a cualquier servidor. Durante la fase de modelado, se establecieron patrones de arquitectura como el Modelo-Vista-Controlador (MVC) y el diseño *Top-Down*. No obstante, el desarrollo no estuvo exento de dificultades; a medida que se implementaron soluciones de concurrencia, surgieron problemas severos de legibilidad que dificultaban la identificación de funciones en el código, requiriendo una refactorización completa de los archivos en ambos componentes. A lo largo de este reporte se detallará de forma exhaustiva el modelado del sistema, la justificación detrás de cada decisión arquitectónica, la implementación técnica de las herramientas seleccionadas y las conclusiones obtenidas de esta experiencia integral de desarrollo de software.

2. Modelado y Programación

Para iniciar con la realización de este chat, primero se tuvo que realizar una ardua investigación sobre las herramientas de forma detallada. Además, desde un inicio se nos notificó que estaba estrictamente prohibido utilizar el lenguaje de programación *Java*, lo cual resultó un pequeño impedimento ya que era la única herramienta sobre la que se poseía un dominio. Por ello, se inició la búsqueda de alternativas tanto para el cliente como para el servidor.

Después de investigar y evaluar varias opciones, se decidió utilizar el lenguaje de programación **Rust** para el servidor y el toolkit **Flutter** (que utiliza el lenguaje *Dart*) para el cliente y la interfaz gráfica. Se optó por Rust debido a su forma peculiar de manejar la memoria, conocida como *Rust Ownership*: Rust asigna cada dato en el heap a una variable propietaria única; cuando esa variable sale de su ámbito (scope), Rust libera automáticamente la memoria asociada, previniendo fugas de memoria y punteros colgados sin necesidad de gestión manual. Esto permite que Rust no dependa de un recolector de basura (Garbage Collector) como ocurre en Java [3].

Por su parte, Flutter es un framework *open-source* desarrollado por Google en 2015 con el objetivo de construir interfaces gráficas y compilarlas de forma nativa en múltiples plataformas. Su paradigma es Orientado a Objetos (OOP) y la sintaxis que maneja es muy similar a la de Java, por lo que resultó ideal para el desarrollo del cliente. Al ser multiplataforma, se optó por compilar únicamente para Linux, macOS y Windows, ya que este chat será un programa exclusivo para entornos de escritorio. La configuración no fue un problema, pues el propio constructor de Flutter brinda las herramientas necesarias para estos sistemas operativos.

Una vez definidos los sistemas operativos, se continuó con el diseño estructural del proyecto. Se investigaron varios patrones de arquitectura de software, destacando el patrón Modelo-Vista-Controlador (MVC), la arquitectura Cliente-Servidor y el diseño Top-Down. Inicialmente, se optó por trabajar con el patrón MVC y una arquitectura Cliente-Servidor. Sin embargo, a medida que el proyecto avanzaba y se implementaban patrones de diseño de *conurrencia*, surgieron problemas de legibilidad; llegó un momento en el que era muy difícil identificar qué parte del código representaba cada función. Por esto, se implementó

una refactorización completa de los archivos, tanto en el servidor como en el cliente.

Teniendo esto en cuenta, procederemos a explicar el servidor de Rust.

2.1. Servidor en Rust

Consideramos que el servidor es probablemente una de las partes menos triviales de implementar del proyecto. Esto se debe a que requiere la ejecución de acciones de forma asíncrona; es decir, el servidor debe ser capaz de realizar varias tareas al mismo tiempo de manera que las acciones bloqueantes puedan pausarse y permitir que el sistema procese otras peticiones.

La organización final de los archivos se basó en el patrón de diseño "Top-Down". Se partió de una carpeta general llamada *rust_server* generada con la ayuda de la herramienta Cargo. Dentro de esta estructura, la carpeta *src* actúa como la raíz de todo el servidor. En su interior se crearon dos subcarpetas: *salas* y *usuarios*, las cuales adoptaron todos los *enums* creados en el archivo de *protocolo*. En estas subcarpetas se agregaron los algoritmos que son utilizados por el *manejador*. Esto se puede ver de forma más clara en el siguiente diagrama de componentes.

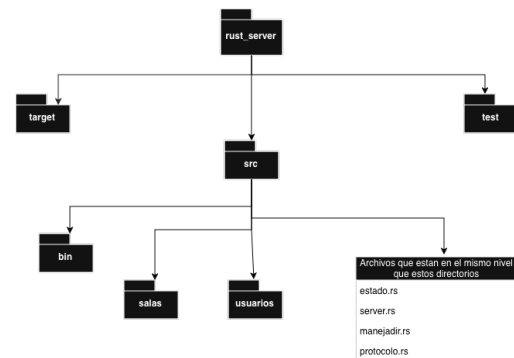


Figura 1: Diagrama de estructura del servidor en Rust (se omiten algunos archivos auxiliares para mayor claridad).

En la parte del diseño se detallará de forma más explícita el funcionamiento del servidor, pero en resumen, este debe cumplir con los siguientes puntos:

1. Establecer conexiones de tal manera que se puedan mantener activas por un tiempo indeterminado.

2. Recibir peticiones del cliente y procesarlas estrictamente de acuerdo con el protocolo establecido.
3. Almacenar los usuarios y el estado de las acciones dictadas por el protocolo.
4. Transmitir las acciones y mensajes necesarios al resto de los clientes conectados.
5. Estar programado de tal forma que sea tolerante a fallos (prácticamente indestructible ante desconexiones abruptas).
6. Realizar todas las acciones de red y lectura/escritura de forma asíncrona y concurrente.

El protocolo mencionado nos fue proporcionado desde el inicio como una guía y estándar, garantizando que cualquier cliente de la clase pudiera conectarse a cualquier servidor. Cabe destacar que el servidor representa la mitad de la arquitectura "Cliente-Servidor", un modelo donde múltiples clientes solicitan servicios o recursos de un nodo centralizado [1].

2.2. Cliente en Flutter

Para analizar al cliente, debemos retomar el patrón de diseño Modelo-Vista-Controlador (MVC). De acuerdo con el glosario de MDN Web Docs, este patrón se define de la siguiente manera [2]:

MVC (Modelo-Vista-Controlador) es un patrón en el diseño de software comúnmente utilizado para implementar interfaces de usuario, datos y lógica de control. Enfatiza una separación entre la lógica de negocios y su visualización. Esta "separación de preocupaciones" proporciona una mejor división del trabajo y una mejora de mantenimiento. Las tres partes del patrón de diseño de software MVC se pueden describir de la siguiente manera:

1. **Modelo:** Maneja datos y lógica de negocios.
2. **Vista:** Se encarga del diseño y presentación.
3. **Controlador:** Enruta comandos a los modelos y vistas.

Bajo esta filosofía, el cliente desarrollado debe cumplir con las siguientes responsabilidades:

1. Conectarse al servidor de forma exitosa a través de un socket TCP.
2. Empaquetar y enviar los comandos ingresados de forma que coincidan exactamente con lo esperado por el servidor.
3. Traducir y deserializar los mensajes recibidos desde el servidor para actualizar la interfaz.
4. Proporcionar una Interfaz de Usuario (UI) intuitiva, aislando al usuario final de la complejidad de la red.

El objetivo principal de la interfaz gráfica es que el usuario final pueda comunicarse utilizando "lenguaje natural", mientras el cliente se encarga de traducir estas interacciones en los comandos estructurados (en formato JSON) necesarios para la comunicación con el servidor. De esta manera, el usuario interactúa cómodamente sin tener que preocuparse por tareas técnicas como escribir JSONs a mano o leer logs en una terminal.

2.3. Diseño general

Para poder entender de mejor manera el como funciona el servidor, se realizó el siguiente diagrama de secuencia:

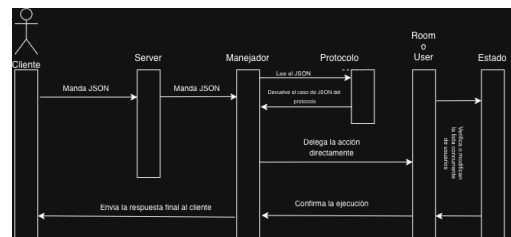


Figura 2: Diagrama de Secuencia del servidor en Rust.

Lo que muestra este diagrama es:

1. **Traducción:** *manejador* → *Protocolo* (pide leer el JSON) y luego *Protocolo* → *manejador* (devuelve el dato estructurado).
2. **Enrutamiento:** *manejador* → *Room o User* (delega la acción directamente, sin involucrar al protocolo).
3. **Memoria:** *Room o User* ↔ *Estado* (verifican o modifican la lista concurrente de usuarios).

4. **Retorno:** *Room* o *User* → *manejador* (confirman la ejecución).
5. **Salida:** *manejador* → *Cliente* (envía la respuesta final a través de la red).

Con esto nos damos una idea más clara de como es que el servidor debe de funcionar para que se cumpla con los requerimientos que fueron especificados.

Ahora, para poder entender como es que funciona el Cliente, se utilizará un diagrama del patron Modelo-Vista-Controlador

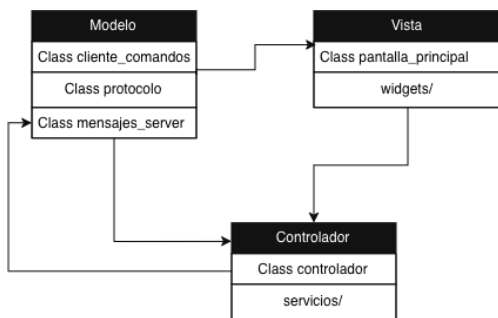


Figura 3: Diagrama de MVC del cliente en Faltter.

El diagrama de clases y paquetes (Figura 3) ilustra la implementación concreta del patrón Modelo-Vista-Controlador en el cliente de Flutter. La estructura garantiza una separación de responsabilidades, dividiendo el sistema en tres capas principales:

1. **Vista:** Compuesta por la clase `pantalla_principal` y la subcarpeta de `widgets/`. Esta capa es puramente visual y declarativa. Se encarga de renderizar la interfaz de usuario y capturar los eventos físicos (como el ingreso de texto o el clic en botones). Como indica el flujo, la Vista no toma decisiones lógicas; simplemente delega las acciones del usuario enviándolas hacia el Controlador.
2. **Controlador:** Actúa como el cerebro y mediador de la aplicación. Agrupa la clase `controlador` y la subcarpeta de `servicios/` (donde reside el manejo del socket TCP). Su función es recibir las interacciones de la Vista, ejecutar la lógica de red para comunicarse con el servidor en Rust y, una vez que obtiene una respuesta, procesar esos datos para enviarlos al Modelo.

3. **Modelo:** Contiene las clases `cliente_comandos`, `protocolo` y `mensajes_server`. Esta capa define las estructuras de datos estrictamente tipadas y representa el estado interno del cliente. Aquí se almacena la información estructurada que coincide con el protocolo del servidor (historial de mensajes, lista de usuarios, etc.).

El ciclo de vida de un evento comienza en la Vista, fluye hacia el Controlador para su procesamiento (envío y recepción por red) y desciende al Modelo para actualizar el estado de los datos. Finalmente, el ciclo se cierra cuando el cambio de estado en el Modelo notifica a la Vista, provocando que la interfaz gráfica se reconstruya para mostrar al usuario la información más reciente.

Dentro del proyecto en la carpeta de `cliente_flutter` es posible observar la implementación de este patrón de diseño, sin embargo tambien se utilizó un poco del patron de "Top-Down", especificamntne en la carpeta de vista, ya que por las mismas razones que el servidor, llegó un punto el cual el código era ilegible, y era muy difícil encontrar posibles bugs o errores de la lógica de la interfaz gráfica, es por eso que creo la subcarpeta de `widgets/`, el cual engloba los puntos más primordiales para la interfaz gráfica, haciendo que el archivo de `pantalla_principal` sea mucho más legible.

3. Implementación

Ya una vez definido los patrones de diseño y como será el funcionamiento del servidor y del cliente, procedemos con la implementación de estas partes del proyecto.

3.1. Para el server de Rust

Para el desarrollo cmo tal del server de rust, se opto por utilizar herramientas de programación asíncrona, de forma particular, se hizo uso de la herramienta de Tokio. Tokio es una herramienta diseñada para tranajos asíncronos dentro de Rust, provee las herramientas necesarias para aplicaciones que requieran uso de la web (como es el caso de nuestro servidor), cosas que tiene este runtime a comparación de, la libreria standard, por decir algún ejemplo es que Tokio nos da la capacidad de:

- **Es Rápido:** El runtime de Tokio, es escrito plenamente en Rust, por lo que hereda la propia filosofía de Rust, además de ser escrito sobre el `async` y `await` funciones, las cuales permiten aumentar el número de operaciones concurrentes que se estén realizando, es decir, permite escalar el server a muchos clientes
- **Es Fácil:** A pesar de lo complicado que puede llegar a ser escribir buen código en Rust, Tokio, sigue el nombramiento de varios comandos de la biblioteca `std`, por lo que es fácil convertir código de la librería `std` a Tokio
- **Es Flexible:** Tokio ofrece múltiples variantes de su entorno de ejecución (`*runtime*`), que abarcan desde un entorno multihilo con estrategia de `*work-stealing*` hasta uno ligero de un solo hilo.

Además de utilizar el crate de `serde_json`, la cual nos brinda todas las herramientas necesarias para poder hacer el parseo con el JSON que marca el protocolo, gracias a la documentación fue que nació la idea de agrupar todos los posibles casos del JSON dentro de enums, esto para que fuera más fácil el parseo y no hubiera errores de tipo o de formato.

Otra de las tecnologías que se utilizaron es el uso de `Arc` y de `Mutex`. `Arc` hace referencia a `Atomic Reference Counting`, este se encuentra dentro de la biblioteca estándar, este módulo permite a varios hilos compartir el `ownership` de los datos. Asegurándose que la información solo sea desechada cuando todas las referencias a través de todos los hilos de ejecución hayan terminado. Mientras que `Mutex`, el cual hace referencia a `"Mutual Exclusion"`, es un primitivo de sincronización en `std::sync`, el cual se encarga de que un hilo de ejecución solamente pueda acceder a la información a la vez. Las ventajas de ocupar `Arc` y `Mutex` son:

- Comparte la información de forma segura gracias a `Arc`
- Controla la mutabilidad con `Mutex`
- Elimina data races en tiempo de compilación

Entonces, en el server de Rust podemos recuperar el uso de tecnologías como Rust, `serde_json`, `Arc` y `Mutex`, los cuales considero que fueron las tecnologías más importantes a la hora de realizar

el server. También cabe recalcar que se implementó el uso de pruebas unitarias en el parseo del JSON dentro del servidor y de la realización de una prueba de estrés, ya que es de suma importancia saber que nuestro servidor funciona completamente, y que no tiene defectos que pudieran echar a perder el chat en una situación de la vida real.

En el repositorio de GitHub, al inicio se muestran los comandos necesarios para poder correr el servidor de forma correcta. Además, dentro de la carpeta `rust_server` se encuentra otro `README.md` el cual detalla cómo deben llevarse a cabo las pruebas unitarias y la prueba de estrés, la cual fue escrita con las mismas herramientas con las que se desarrolló todo el servidor. Sin embargo, en este documento se volverá a especificar qué comandos deben ejecutarse para poder correr el servidor:

1. **Requisitos previos:** Es necesario tener instalados Rust y Cargo. Para comprobar que se cuenta con el compilador del lenguaje y el gestor de paquetes, se debe escribir en la terminal:

```
rustc --version && cargo --version
```

2. **Compilación:** Una vez clonado el repositorio, se debe navegar hasta el directorio del servidor y compilar el código mediante el comando:

```
cargo build
```

3. **Ejecución del servidor:** Para correr el programa y levantar el servidor, basta con utilizar `cargo run`. Sin embargo, para poder visualizar todos los registros (`logs`) de las peticiones que recibirá el servidor, se recomienda el uso del siguiente comando, donde se puede especificar el puerto (por defecto se utiliza el 1234):

```
RUST_LOG=info cargo run -- --puerto <PUERTO>
```

4. **Conexión manual (opcional):** Para conectarse al servidor sin necesidad de utilizar el cliente de Flutter, se puede emplear `nc` (Netcat) o `telnet` con la dirección IP de la red local, usando el comando:

```
telnet <DIRECCION_IP> <PUERTO>
```

5. **Pruebas de estrés:** Si se desea ejecutar la prueba de estrés, es necesario aumentar el límite de descriptores de archivos del sistema

operativo para permitir múltiples conexiones simultáneas, y luego correr el entorno de pruebas con los siguientes comandos:

```
ulimit -n 4096
RUST_LOG=info cargo test --test
test_estres -- --nocapture
```

3.2. Para el cliente realizado en Flutter

Para el desarrollo del cliente dentro de Flutter, la realidad es que fue mucho más sencillo de lo esperado, ya que muchas de las herramientas que se necesitaban para poder realizar programación asíncrona y demás ya se encontraban en las bibliotecas que vienen por defecto dentro de Flutter, ya que como se mencionó anteriormente, este framework está diseñado para poder crear aplicaciones en distintos dispositivos que muy probablemente necesiten acceso a conexiones ya sea como nuestro caso, TCP. Con esto dicho, el mayor de las complicaciones fue el uso de la biblioteca de material.dart, ya que esta biblioteca fue la que nos permitió crear nuestra interfaz gráfica, aunque sencilla, es más que funcional, para esta biblioteca se tuvo que consultar varios blogs de internet donde explicaban que es lo que hacía casi que cada comando, al final se llegó a cierta repetición de patrones por lo que llegó un momento en el que el escribir un botón se volvió algo demasiado monótono, sin embargo, cabe recalcar como punto de mejora para este y futuros proyectos, que hay que tener muy en claro que tipo de dispositivos se requerirá utilizar, ya que Flutter también acepta compatibilidad con sistemas móviles como Android o iOS, por lo que se tiene que revisar la documentación para saber que tanto cambia, con solo desarrollar para escritorios o laptop.

Por otro lado, para la configuración y ejecución del cliente gráfico desarrollado en Flutter, se establecen dos modalidades de uso: el modo de desarrollo y la compilación de un ejecutable nativo. A continuación se detallan los pasos necesarios:

1. **Requisitos previos:** Es indispensable contar con el framework de Flutter instalado y configurado según el sistema operativo (distribución de Linux, macOS o Windows). Para verificar que el SDK está correctamente instalado, se puede ejecutar en la terminal:

```
flutter --version
```

2. **Consideración importante:** Antes de iniciar cualquier instancia del cliente, el servidor en Rust **debe estar en ejecución** y escuchando en el puerto designado.

3. **Ejecución en Modo Desarrollador:** Para correr el cliente directamente desde el código fuente para realizar pruebas, se debe navegar al directorio `cliente_flutter` y ejecutar el siguiente comando, sustituyendo los parámetros por la información de la conexión:

```
flutter run -d <OS> -a <IP> -a <
Puerto> -a <Usuario>
```

Nota: Si se desea conectar múltiples clientes simultáneamente desde la misma computadora, basta con abrir diferentes ventanas de terminal y ejecutar el mismo comando, asegurándose de utilizar un nombre de usuario distinto para cada instancia.

4. **Compilación de Ejecutable (Modo Producción):** Si se desea generar una aplicación nativa, primero se debe compilar el proyecto mediante el comando:

```
flutter build <OS>
```

Una vez finalizada la construcción, se puede lanzar el binario generado pasando los argumentos de conexión de forma directa. La ruta del ejecutable final varía dependiendo de la plataforma de destino.

Para entorno macOS:

```
./build/macos/Build/Products/
Release/cliente_flutter <IP>
<Puerto> <Usuario>
```

Para entorno Linux (generalmente):

```
./build/linux/x64/release/bundle/
cliente_flutter <IP> <Puerto>
<Usuario>
```

4. Conclusiones

En términos generales, el desarrollo de este proyecto representó un reto integral, abarcando desde la concepción de una arquitectura desde cero hasta la toma de decisiones críticas sobre las herramientas tecnológicas a utilizar. Como primer gran proyecto de la materia, supuso un excelente acercamiento al ciclo de vida del desarrollo de

software con aplicaciones del mundo real. Aunque el producto final se encuentra en una etapa temprana, la experiencia resultó sumamente desafiante y enriquecedora en el ámbito académico y profesional.

Un aprendizaje fundamental durante este proceso fue la gestión del tiempo y la capacidad de autoestudio. En el desarrollo de software es común enfrentarse a una curva de aprendizaje pronunciada, donde el exceso de investigación puede generar un estancamiento práctico. La rápida evolución de las herramientas exige que los estudiantes y profesionales de la computación desarrollen la habilidad de aprender sobre la marcha y adaptarse continuamente; de lo contrario, el conocimiento técnico queda obsoleto con rapidez.

Asimismo, este proyecto destacó la vital importancia del trabajo colaborativo. Enfrentar problemas técnicos complejos de forma colectiva, apoyándose en el conocimiento y la experiencia de los compañeros de clase, demostró ser una herramienta invaluable. En una industria que en ocasiones fomenta el aislamiento y la individualidad, desarrollar redes de apoyo y un sentido de comunidad facilita enormemente la resolución de problemas lógicos y fortalece las habilidades interpersonales.

Finalmente, el sistema Cliente-Servidor construido posee una base sólida con un alto potencial de escalabilidad. Como trabajo futuro, se podrían proponer diversas expansiones inspiradas en aplicaciones de mensajería modernas, tales como:

- Implementación de fotos de perfil vinculadas a cada usuario.
- Soporte para el envío de archivos multimedia, incluyendo imágenes, notas de voz o fragmentos de video.

Para lograr estas implementaciones, se reconoce que sería indispensable realizar modificaciones sustanciales al protocolo de comunicación, adaptando la serialización JSON y la lógica de ambos extremos (cliente y servidor) para que sean capaces de procesar, transmitir y renderizar este nuevo volumen de datos estructurados.

Referencias

[1] GeeksforGeeks. Client-server architecture - system design, 2024. Accedido: 2024.

[2] Mozilla Developer Network. Mvc (model-view-controller) - mdn web docs glossary, 2024. Accedido: 2024.

[3] Rust Foundation. Understanding ownership - the rust programming language, 2024. Accedido: 2024.